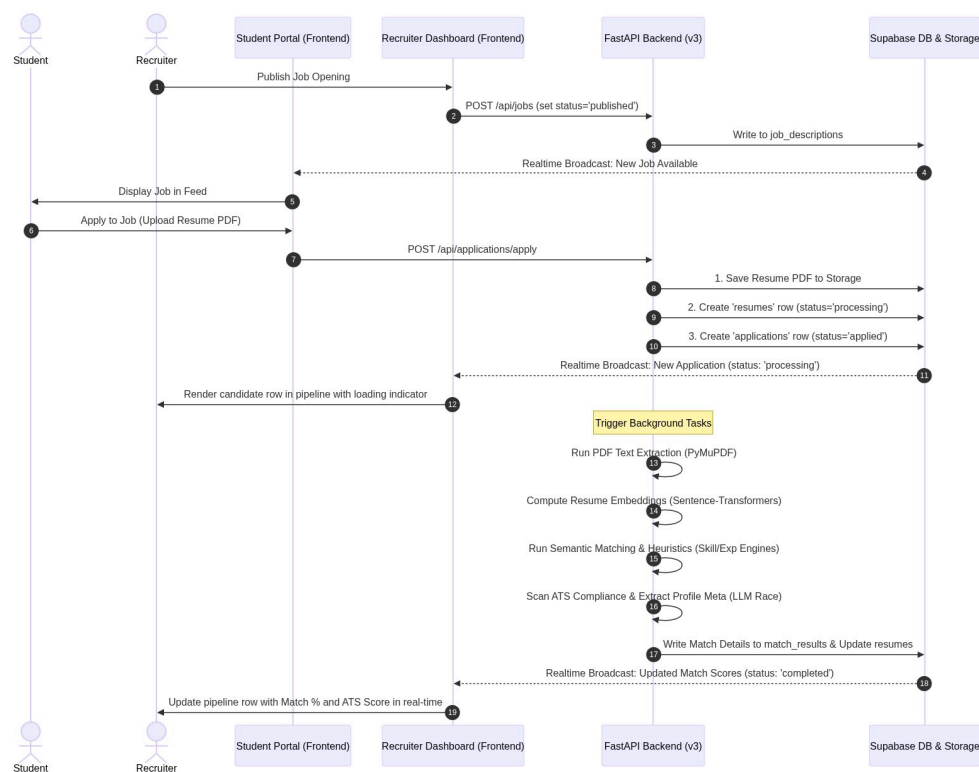


Architectural Design: Real-time Recruiter-to-Student Portal Integration

This document outlines the proposed database models, backend processing pipelines, and real-time synchronization strategy required to connect the HireSense Recruiter Platform with a future Student Job Portal.

System Architecture Overview

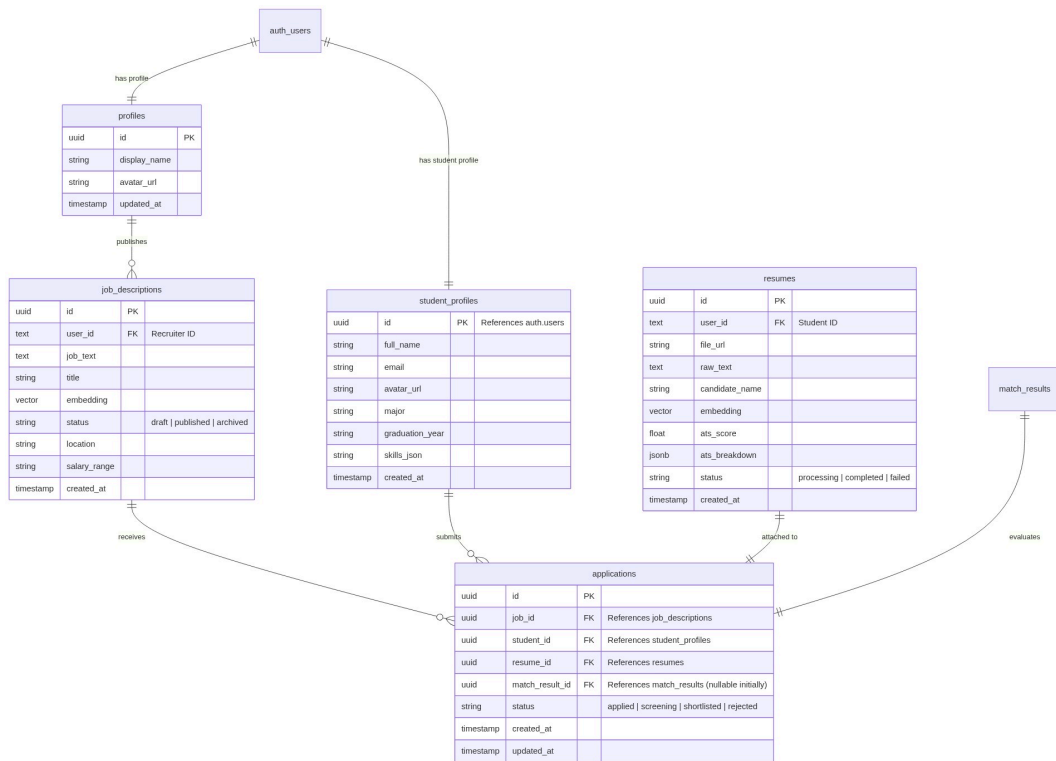
The integration relies on a bi-directional data flow using Supabase Realtime (PostgreSQL Replication). When a student applies, the backend automatically triggers the AI parsing and semantic scoring engine in the background, updating the recruiter's dashboard dynamically as processing stages complete.



Database Schema Extensions

To support the Student Portal and application tracking, we need to transition from a single-tenant "recruiter-only upload" schema to a multi-role relational model.

Entity Relationship Diagram



Proposed SQL Migrations

-- 1. Create Student Profiles table

```
CREATE TABLE IF NOT EXISTS public.student_profiles (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    full_name TEXT NOT NULL,
    email TEXT NOT NULL,
    avatar_url TEXT,
    major TEXT,
    graduation_year TEXT,
    skills_json JSONB DEFAULT '[]',
    created_at TIMESTAMPTZ WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, now()) NOT NULL
);
```

-- 2. Extend Job Descriptions table with job board meta

```
ALTER TABLE public.job_descriptions
    ADD COLUMN IF NOT EXISTS status TEXT NOT NULL DEFAULT 'published', -- 'draft',
    'published', 'archived'
    ADD COLUMN IF NOT EXISTS location TEXT,
    ADD COLUMN IF NOT EXISTS salary_range TEXT;
```

-- 3. Create Applications table

```
CREATE TABLE IF NOT EXISTS public.applications (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    job_id UUID REFERENCES public.job_descriptions(id) ON DELETE CASCADE NOT NULL,
    student_id UUID REFERENCES public.student_profiles(id) ON DELETE CASCADE NOT NULL,
    resume_id UUID REFERENCES public.resumes(id) ON DELETE SET NULL,
    match_result_id UUID REFERENCES public.match_results(id) ON DELETE SET NULL,
    status TEXT NOT NULL DEFAULT 'applied', -- 'applied', 'screening', 'shortlisted',
    'rejected'
    created_at TIMESTAMPTZ WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, now()) NOT NULL,
    updated_at TIMESTAMPTZ WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, now()) NOT NULL
);
```

```
);

-- 4. Enable Supabase Realtime Replication for critical tables
ALTER PUBLICATION supabase_realtime ADD TABLE public.applications;
ALTER PUBLICATION supabase_realtime ADD TABLE public.resumes;
ALTER PUBLICATION supabase_realtime ADD TABLE public.match_results;
ALTER PUBLICATION supabase_realtime ADD TABLE public.job_descriptions;

-- 5. Performance Indexes
CREATE INDEX IF NOT EXISTS idx_applications_job_id ON public.applications(job_id);
CREATE INDEX IF NOT EXISTS idx_applications_student_id ON public.applications(student_id);
CREATE INDEX IF NOT EXISTS idx_job_descriptions_status ON public.job_descriptions(status);
```

Backend Process Design (FastAPI)

1. Application Endpoint

A student submits their application by calling a secure endpoint containing their resume file and the target job ID.

```
# Route: POST /api/applications/apply
# Flow:
# 1. Verify student JWT (extract student_id).
# 2. Upload resume PDF to Supabase Storage ('resumes' bucket).
# 3. Create a placeholder resumes row:
#     status = 'processing'
# 4. Create an applications row:
#     job_id = job_id, student_id = student_id, resume_id = resume_id, status = 'applied'
# 5. Fire BackgroundTask: process_student_application(application_id, resume_id, job_id,
raw_pdf_bytes)
# 6. Return 201 Response: {"application_id": id, "status": "applied"}
```

2. Async Background Screening Engine

The background worker extracts text, parses keywords, calculates similarities, and saves the matching report. Because database writes are listened to in real-time, every table change automatically pushes progress notifications to the recruiter's UI.

```
async def process_student_application(app_id: str, resume_id: str, job_id: str, file_bytes:
bytes):
    db = get_admin_db() # Use clean client
    try:
        # Step A: Parse PDF text
        raw_text = extract_text(file_bytes)

        # Step B: Generate dense vector embedding (Sentence-Transformers)
        vector = generate_embedding(raw_text)

        # Step C: Parse ATS compliance and candidate details (NIM Racing Engine)
        ats_report = await scan_ats_compliance(raw_text)
        candidate_name = ats_report.get("candidate_name")

        # Update resume record in DB (Fires first Realtime update)
```

```

db.table("resumes").update({
    "raw_text": raw_text,
    "embedding": vector,
    "ats_score": ats_report.get("score", 0),
    "ats_breakdown": json.dumps(ats_report.get("breakdown", [])),
    "candidate_name": candidate_name,
    "status": "completed"
}).eq("id", resume_id).execute()

# Step D: Load Job details & calculate matching metrics
job = db.table("job_descriptions").select("job_text").eq("id",
job_id).execute().data[0]

# Run sub-engines
skill_score = calculate_skill_overlap(raw_text, job["job_text"])
exp_score = calculate_experience_score(raw_text, job["job_text"])
strength = compute_strength(raw_text)

# Semantic Cosine Similarity
job_emb = db.table("job_descriptions").select("embedding").eq("id",
job_id).execute().data[0]["embedding"]
semantic_sim = float(cosine_similarity([np.array(job_emb)],
[ np.array(vector)])[0][0])

# Calculate final weighted match percentage
final_score = (semantic_sim * 0.40) + (skill_score * 0.30) + (exp_score * 0.20) +
((strength/100) * 0.10)
risk_level = get_risk_level(final_score)

# Step E: Insert match_results (Fires second Realtime update)
match_res = db.table("match_results").insert({
    "resume_id": resume_id,
    "job_id": job_id,
    "semantic_score": round(semantic_sim, 4),
    "skill_score": round(skill_score, 4),
    "experience_score": round(exp_score, 4),
    "final_score": round(final_score, 4),
    "resume_strength": strength,
    "risk_level": risk_level,
    "candidate_status": "pending"
}).execute()
match_id = match_res.data[0]["id"]

# Step F: Connect match ID to the application (Fires final Realtime update)
db.table("applications").update({
    "match_result_id": match_id,
    "status": "screening"
}).eq("id", app_id).execute()

except Exception as e:
    print(f"Failed to process application {app_id}: {e}")
    db.table("resumes").update({"status": "failed"}).eq("id", resume_id).execute()
    db.table("applications").update({"status": "failed"}).eq("id", app_id).execute()

```

Real-time Synchronization Strategy

We can use Supabase Realtime JavaScript SDK to manage subscriptions. Since Supabase broadcasts PostgreSQL transaction changes instantly, we don't need to spin up WebSockets/SSE on the FastAPI side.

Recruiter UI Integration (React)

In the Recruiter's Job Application dashboard, the component subscribes to modifications on the applications, resumes, and match_results tables filtered by the current job_id:

```
import { useEffect, useState } from 'react';
import { supabase } from '../lib/supabase';

export function useJobApplications(jobId) {
  const [applications, setApplications] = useState([]);

  useEffect(() => {
    // 1. Initial Data Fetch
    fetchApplications();

    // 2. Setup Realtime Listener
    const channel = supabase
      .channel(`job-applications-${jobId}`)
      .on(
        'postgres_changes',
        { event: '*', schema: 'public', table: 'applications', filter:
`job_id=eq.${jobId}` },
        (payload) => {
          handleApplicationChange(payload);
        }
      )
      .on(
        'postgres_changes',
        { event: 'UPDATE', schema: 'public', table: 'resumes' },
        (payload) => {
          handleResumeUpdate(payload);
        }
      )
      .on(
        'postgres_changes',
        { event: 'INSERT', schema: 'public', table: 'match_results' },
        (payload) => {
          handleMatchResultInsert(payload);
        }
      )
      .subscribe();

    return () => {
      supabase.removeChannel(channel);
    };
  }, [jobId]);

  const fetchApplications = async () => {
    // Fetches applications, joining student profile, resume status, and match results
  };
}
```

```

const handleApplicationChange = (payload) => {
  if (payload.eventType === 'INSERT') {
    // Instantly append new applicant card with a "Processing..." status
    setApplications(prev => [payload.new, ...prev]);
  } else if (payload.eventType === 'UPDATE') {
    // Update application status (e.g. from 'applied' to 'screening')
    setApplications(prev => prev.map(app => app.id === payload.new.id ? { ...app,
...payload.new } : app));
  } else if (payload.eventType === 'DELETE') {
    setApplications(prev => prev.filter(app => app.id !== payload.old.id));
  }
};

const handleResumeUpdate = (payload) => {
  // Dynamic state update when parsing is completed
  setApplications(prev => prev.map(app =>
    app.resume_id === payload.new.id
      ? { ...app, resume_status: payload.new.status, ats_score:
payload.new.ats_score, candidate_name: payload.new.candidate_name }
      : app
  ));
};

const handleMatchResultInsert = (payload) => {
  // Connect match scores (Match %, Risk Level) to candidate card dynamically
  setApplications(prev => prev.map(app =>
    app.resume_id === payload.new.resume_id
      ? { ...app, match_score: payload.new.final_score, risk_level:
payload.new.risk_level }
      : app
  ));
};

return { applications };
}

```